Unit – 4

Why Python?

Python is a high-level, interpreted, general-purpose programming language. It is widely used because of its simple syntax, readability, and versatility.

Applications of Python:

1). Artificial Intelligence (AI)

- Used to develop intelligent systems.
- Used in machine learning and deep learning.
- Used in chatbots and virtual assistants.
- **Example:** ChatGPT, voice assistants.

2). Data Science

- Used to collect, process, and analyze large amounts of data.
- Used for data visualization and statistical analysis.
- Helps in predicting future trends.
- **Libraries:** NumPy, Pandas, Matplotlib.

3). Web Development

- Used to develop dynamic websites and web applications.
- Used for server-side programming.
- Helps in database connectivity and backend development.
- **Frameworks:** Django, Flask.

4). Automation

- Used to automate repetitive tasks.
- Used for file handling and data entry.
- Used for sending automated emails and web scraping.
- Helps save time and reduce human errors.

Variables

A variable is a name used to store data in a program. In Python, a variable is created when a value is assigned to it. No separate declaration is required.

Syntax:

variable_name = value

Example:

```
a = 10
b = 20
name = "Ankit"
print(a)
print(b)
print(name)
```

Output:

```
10
20
Ankit
```

Rules for Naming Variables:

1. **1).** A variable name must start with a letter or underscore (_).
2. **2).** A variable name cannot start with a digit.
3. **3).** A variable name can contain letters, digits and underscores.
4. **4).** Spaces are not allowed in variable names.
5. **5).** Python keywords cannot be used as variable names.
6. **6).** Variable names are case-sensitive.

Valid Variable Names:

```
name = "Ankit"
age = 20
_student = "Rahul"
total_marks = 450
```

Invalid Variable Names:

```
2name = "Ankit"
my name = "Ankit"
class = 10
```

Data Types

A data type defines the type of value that a variable can store. Python provides different built-in data types to store different kinds of data.

Data Type	Description	Example
int	Stores integer numbers	a = 10
float	Stores decimal numbers	a = 10.5
complex	Stores complex numbers	a = 2+3j
str	Stores text or strings	a = "Python"
bool	Stores True or False	a = True
list	Stores multiple values	a = [10, 20, 30]
tuple	Stores multiple values	a = (10, 20, 30)
set	Stores unique elements	a = {10, 20, 30}
dict	Stores key-value pairs	a = {"name": "Ankit"}

Example Program:

```
a = 10
b = 10.5
c = 2 + 3j
d = "Python"
e = True
f = [10, 20, 30]
g = (10, 20, 30)
h = {10, 20, 30}
i = {"name": "Ankit"}
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
print(type(f))
```

```
print(type(g))

print(type(h))

print(type(i))
```

Output:

```
<class 'int'>

<class 'float'>

<class 'complex'>

<class 'str'>

<class 'bool'>

<class 'list'>

<class 'tuple'>

<class 'set'>

<class 'dict'>
```

print() Function

The print() function is used to display output on the screen. It is one of the most commonly used built-in functions in Python.

Syntax:

```
print(value)
```

Example 1: Printing a String

```
print("Hello World")
```

Output:

```
Hello World
```

Example 2: Printing a Number

```
print(100)
```

Output:

```
100
```

Example 3: Printing Multiple Values

```
a = 10

b = 20
```

print(a, b)

Output:

10 20

Example 4: Printing Variables with Messages

name = "Ankit"

age = 20

print("My name is", name)

print("My age is", age)

Output:

My name is Ankit

My age is 20

input() Function

The input() function is used to accept data from the user through the keyboard.

By default, the input() function returns the entered value as a string.

Syntax:

variable_name = input("Message")

Example 1: Taking String Input

name = input("Enter your name: ")print("Welcome", name)

Output:

Enter your name: Ankit

Welcome Ankit

Example 2: Taking Integer Input

To accept an integer value, we use the int() function with input().

age = int(input("Enter your age: "))

print("Your age is", age)

Example 3: Taking Float Input

To accept a decimal value, we use the float() function.

price = float(input("Enter price: "))

print("Price =", price)

Example 4: Addition of Two Numbers

```
a = int(input("Enter first number: "))

b = int(input("Enter second number: "))

sum = a + bprint("Sum =", sum)
```

Output:

Enter first number: 10

Enter second number: 20

Sum = 30

Operators in Python

An operator is a special symbol or keyword used to perform operations on variables and values.

Python supports the following types of operators:

1. **1).** Arithmetic Operators
2. **2).** Relational or Comparison Operators
3. **3).** Logical Operators
4. **4).** Assignment Operators
5. **5).** Bitwise Operators
6. **6).** Membership Operators
7. **7).** Identity Operators

A. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations such as addition, subtraction, multiplication and division.

Operator	Name	Description	Example	Result
+	Addition	Adds two values	10 + 5	15
-	Subtraction	Subtracts two values	10 - 5	5
*	Multiplication	Multiplies two values	10 * 5	50
/	Division	Returns quotient	10 / 3	3.333...
%	Modulus	Returns remainder	10 % 3	1
//	Floor Division	Returns quotient rounded down	10 // 3	3
**	Exponentiation	Calculates power	2 ** 3	8

Example Program:

```
a = 10
b = 3
print("Addition =", a + b)
print("Subtraction =", a - b)
print("Multiplication =", a * b)
print("Division =", a / b)
print("Modulus =", a % b)
print("Floor Division =", a // b)
print("Power =", a ** b)
```

Output:

Addition = 13

Subtraction = 7

Multiplication = 30

Division = 3.3333333333333335

Modulus = 1

Floor Division = 3

Power = 1000

B. Relational or Comparison Operators

Relational operators are used to compare two values. They return either True or False.

Operator	Name	Example	Result
==	Equal to	10 == 10	True
!=	Not equal to	10 != 5	True
>	Greater than	10 > 5	True
<	Less than	5 < 10	True
>=	Greater than or equal to	10 >= 10	True

$a = 10$

$b = 5$

```
print(a < b)
```

```
print(a >= b)
print(a <= b)
```

Output:

```
False
True
False
```

C. Logical Operators

Logical operators are used to combine two or more conditions. They return Boolean results.

Python has three logical operators:

Operator	Description	Example
and	Returns True if both conditions are True	10 > 5 and 20 > 10
or	Returns True if at least one condition is True	10 < 5 or 20 > 10
not	Reverses the Boolean result	not(10 > 5)

Example Program:

```
a = 10
b = 20
print(a > 5 and b > 10)
print(a > 15 or b > 10)
print(not(a > 5))
```

Output:

```
True
True
False
```

D. Assignment Operators

Assignment operators are used to assign values to variables and update their values.

Operator	Example	Equivalent Expression
=	$a = 10$	Assigns 10 to a
+=	$a += 5$	a = a + 5

$-=$	$a -= 5$	a = a - 5
$*=$	$a *= 5$	a = a * 5
$/=$	$a /= 5$	a = a / 5
$\% =$	$a \% = 5$	a = a % 5
$//=$	$a //= 5$	a = a // 5
$**=$	$a **= 2$	a = a ** 2

Example Program:

a = 10

$a += 5$

print(a)

$a -= 3$

print(a)

$a *= 2$

print(a)

Output:

15

12

24

Difference Between = and ==

= (Assignment Operator)	**== (Comparison Operator)**
Used to assign a value to a variable.	Used to compare two values.
It assigns a value.	It returns True or False.
Example: a = 10	Example: a == 10

Example:

a = 10

print(a == 10)

Output:

True

Control Flow in Python

Control flow refers to the order in which statements are executed in a program. It controls the execution of statements based on conditions and repetitions.

Main control flow statements:

1). Conditional Statements – if, if-else, elif.

2). Looping Statements – for, while.

if Statement

The if statement executes a block of code only when the given condition is True.

Syntax:

```
if condition:
    statement
```

Example:

```
age = 20
if age >= 18:
    print("Eligible to vote")
```

Output:

Eligible to vote

if-else Statement

The if-else statement executes one block when the condition is True and another block when the condition is False.

Syntax:

```
if condition:
    statement1
else:
    statement2
```

Example:

```
num = 10
if num % 2 == 0:
    print("Even Number")
else:
```

```
    print("Odd Number")
```

Output:

Even Number

if-elif-else Statement

It is used to check multiple conditions in a program.

Syntax:

```
if condition1:
    statement1
elif condition2:
    statement2
else:
    statement3
```

Example:

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Output:

Grade B

Nested if Statement

An if statement inside another if statement is called a nested if statement.

Example:

```
age = 20
```

```
if age >= 18:
    if age >= 21:
        print("Eligible")
    else:
        print("Not eligible")
else:
    print("Under age")
```

Output:

Not eligible

for Loop in Python

A for loop is used to execute a block of statements repeatedly for each element in a sequence such as a list, tuple or string.

Syntax:

```
for variable in sequence:
    statements
```

Example 1:

```
for i in range(1, 4):
print(i)
```

Output:

1

2

3

range() Function

The range() function generates a sequence of numbers and is commonly used with the for loop.

Syntax	**Description**
range(5)	Generates 0 to 4
range(1, 6)	Generates 1 to 5
range(1, 10, 2)	Generates 1, 3, 5, 7, 9

Example 2: Sum of First 10 Natural Numbers

```
sum = 0
for i in range(1, 11):
    sum = sum + i
print("Sum =", sum)
```

Output:

Sum = 55

while Loop in Python

A while loop executes a block of statements repeatedly as long as the given condition is True.

Syntax:

```
while condition:
    statements
```

Example 1:

```
i = 1
while i <= 3:
    print(i)
    i = i + 1
```

Output:

1

2

3

Example 2: Sum of First 10 Natural Numbers

```
i = 1
sum = 0
while i <= 10:
    sum = sum + i
    i = i + 1
print("Sum =", sum)
```

Output: Sum = 55

Functions in Python

A function is a block of reusable code that performs a specific task. It helps to reduce code repetition and makes a program easier to understand and maintain.

Python provides two types of functions:

1). **Built-in Functions:** Functions already available in Python, such as print(), input(), len() and type().

2). **User-defined Functions:** Functions created by the programmer using the def keyword.

def Keyword

The def keyword is used to define or create a user-defined function in Python.

Syntax:

```
def function_name(parameters):
    statements
```

Example:

```
def greet():
    print("Hello, Welcome to Python")
greet()
```

Output:

Hello, Welcome to Python

Explanation:

- def is used to define a function.
- greet is the function name.
- () contains parameters.
- The function body contains statements.
- greet() calls the function.

Function with Parameters and Arguments

Parameters are variables defined in a function definition, whereas arguments are actual values passed to the function during the function call.

Example:

```
def add(a, b):
    print("Sum =", a + b)
```

```
add(10, 20)
```

Output:

```
Sum = 30
```

Here, a and b are parameters, whereas 10 and 20 are arguments.

return Statement

The return statement is used to send a result back from a function to the calling statement. It also terminates the function execution.

Syntax:

```
def function_name():
    return value
```

Example:

```
def add(a, b):
    return a + b
result = add(10, 20)
print("Sum =", result)
```

Output:

```
Sum = 30
```

Explanation:

- The function receives two arguments.
- It calculates their sum.
- The return statement sends the result back.
- The result is stored in the variable result.

Note: If a function does not contain a return statement, Python automatically returns None.

Default Arguments

Default arguments are the values assigned to function parameters during function definition. If the user does not provide an argument, the default value is used.

Syntax:

```
def function_name(parameter=value):
    statements
```

Example:

```
def greet(name="Ankit"):
    print("Hello", name)
greet()
greet("Rahul")
```

Output:

Hello Ankit

Hello Rahul

Explanation:

- In the first function call, no argument is passed, so the default value "Ankit" is used.
- In the second function call, "Rahul" is passed, so it replaces the default value.

Example of Default Arguments with Multiple Parameters

```
def student(name, course="B.Tech"):
    print("Name:", name)
    print("Course:", course)
student("Ankit")
student("Rahul", "BCA")
```

Output:

Name: Ankit

Course: B.Tech

Name: Rahul

Course: BCA

Python List, Tuple, Dictionary and Set – Basic Operations

1. List in Python

A list is a collection of elements used to store multiple values in a single variable. A list is ordered, allows duplicate values and is mutable (its elements can be changed after creation).

Syntax:

```
list_name = [element1, element2, element3]
```

Example:

numbers = [10, 20, 30, 40, 50]print(numbers)

Output:

[10, 20, 30, 40, 50]

Basic Operations on List

1. Accessing Elements

Elements of a list can be accessed using their index. Indexing starts from 0.

numbers = [10, 20, 30, 40, 50]print(numbers[0])print(numbers[2])print(numbers[-1])

Output:

10

30

50

2. Adding Elements

Python provides the following methods to add elements to a list.

Method	**Description**
append()	Adds an element at the end.
insert()	Adds an element at a specified position.
extend()	Adds multiple elements from another iterable.

Example:

numbers = [10, 20, 30]

numbers.append(40)

print(numbers)

numbers.insert(1, 15)

print(numbers)

numbers.extend([50, 60])

print(numbers)

Output:

[10, 20, 30, 40]

[10, 15, 20, 30, 40]

[10, 15, 20, 30, 40, 50, 60]

What is Iterable in extend() in List?

An iterable is an object whose elements can be accessed one by one, such as a list, tuple, string or set.

The extend() method adds all elements of an iterable to the end of an existing list.

Syntax:

list.extend(iterable)

Example 1: Using another List

numbers = [10, 20, 30]numbers.extend([40, 50])print(numbers)

Output:

[10, 20, 30, 40, 50]

Example 2: Using a String

letters = ['A', 'B']letters.extend("CD")print(letters)

Output:

['A', 'B', 'C', 'D']

Explanation: The string "CD" is an iterable, so its characters are added separately to the list.

Difference between append() and extend():

a = [10, 20]a.append([30, 40])print(a)b = [10, 20]b.extend([30, 40])print(b)

Output:

[10, 20, [30, 40]]

[10, 20, 30, 40]

3. Removing Elements

Method	Description
remove()	Removes the specified element.
pop()	Removes and returns an element at a specified index.
clear()	Removes all elements from the list.

Example:

numbers = [10, 20, 30, 40]

numbers.remove(20)

print(numbers)

numbers.pop()

print(numbers)

numbers.clear()

print(numbers)

The pop() method removes an element from a list and returns the removed element.

It can remove an element using its index.

Syntax:

list.pop(index)

- If no index is provided, it removes the last element.
- If an index is provided, it removes the element at that position.
- It returns the removed element.

Example 1: Without Index

numbers = [10, 20, 30, 40]x = numbers.pop()print("Removed element:", x)print("Updated list:", numbers)

Output:

Removed element: 40

Updated list: [10, 20, 30]

Explanation: Since no index is given, pop() removes the last element, 40, and stores it in variable x.

Example 2: With Index

numbers = [10, 20, 30, 40]

x = numbers.pop(1)

print("Removed element:", x)

print("Updated list:", numbers)

Output:

Removed element: 20

Updated list: [10, 30, 40]

Explanation: Index 1 represents the second element, so 20 is removed and returned.

Output:

[10, 30, 40]

[10, 30]

[]

4. Updating Elements

List elements can be modified by assigning a new value to a particular index.

Example:

numbers = [10, 20, 30]

numbers[1] = 50

print(numbers)

Output:

[10, 50, 30]

5. Slicing a List

List slicing is used to access a particular portion of a list.

Syntax:

list[start : stop : step]

There are three parts:

1. Start: The index from which slicing begins. It is included in the result.
2. Stop: The index where slicing ends. It is not included in the result.
3. Step: Specifies how many positions to move forward or backward between selected elements.

Example:

numbers = [10, 20, 30, 40, 50, 60, 70]

print(numbers[1:5:1])

print(numbers[1:5:2])

print(numbers[::2])

print(numbers[::-1])

Output:

[20, 30, 40, 50]

[20, 40]

[10, 30, 50, 70]

[70, 60, 50, 40, 30, 20, 10]

Explanation:

- numbers[1:5:1]: Starts at index 1, stops before index 5, and moves one position at a time.
- numbers[1:5:2]: Starts at index 1 and moves two positions at a time.
- numbers[::2]: Selects every second element from the beginning to the end.
- numbers[::-1]: Reverses the list by moving backward.

Important: If start or stop is omitted, Python uses the beginning or end of the list as appropriate.

6. Other Common List Functions

Function	**Description**
len()	Returns the number of elements.
sort()	Sorts elements in ascending order by default.
reverse()	Reverses the order of elements.
index()	Returns the index of an element.
count()	Returns the number of occurrences of an element.

Example:

```
numbers = [30, 10, 20, 10]

print(len(numbers))

print(numbers.count(10))

print(numbers.index(20))

numbers.sort()

print(numbers)

numbers.reverse()

print(numbers)
```

Output:

4

2

2

[10, 10, 20, 30]

[30, 20, 10, 10]

2. Tuple in Python

A tuple is an ordered collection of elements used to store multiple values in a single variable. A tuple allows duplicate values but is immutable, which means its elements cannot be changed after creation.

Syntax:

tuple_name = (element1, element2, element3)

Example:

numbers = (10, 20, 30, 40, 50)

print(numbers)

Output:

(10, 20, 30, 40, 50)

Basic Operations on Tuple

1. Accessing Elements

Tuple elements can be accessed using their index. Indexing starts from 0.

Example:

numbers = (10, 20, 30, 40, 50)

print(numbers[0])

print(numbers[2])

print(numbers[-1])

Output:

10

30

50

2. Slicing a Tuple

Slicing is used to access a specific portion of a tuple.

Example:

numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])

print(numbers[:3])

print(numbers[2:])

Output:

(20, 30, 40)

(10, 20, 30)

(30, 40, 50)

3. Concatenation

Concatenation is used to combine two or more tuples using the + operator.

Example:

a = (10, 20, 30)

b = (40, 50, 60)

c = a + bprint(c)

Output:

(10, 20, 30, 40, 50, 60)

4. Repetition

The * operator is used to repeat the elements of a tuple.

Example:

numbers = (10, 20)

print(numbers * 3)

Output:

(10, 20, 10, 20, 10, 20)

5. Tuple Methods

Method/Function	**Description**
len()	Returns the number of elements.
count()	Returns the number of occurrences of an element.
index()	Returns the index of an element.
max()	Returns the largest element.
min()	Returns the smallest element.

Example:

numbers = (10, 20, 30, 20, 40)

```
print(len(numbers))
print(numbers.count(20))
print(numbers.index(30))
print(max(numbers))
print(min(numbers))
```

Output:

5

2

2

40

10

6. Packing and Unpacking

- Packing: Assigning multiple values to a single tuple.
- Unpacking: Assigning tuple elements to multiple variables.

Example:

```
student = ("Ankit", 20, "B.Tech")
name, age, course = student
print(name)
print(age)
print(course)
```

Output:

Ankit

20

B.Tech

Note: Tuple elements cannot be directly added, removed or modified after creation.

3. Dictionary in Python

A dictionary is a collection of data stored in the form of key-value pairs. Each key is used to access its corresponding value. Dictionaries are mutable and keys must be unique.

Syntax:

dictionary_name = { "key1": "value1", "key2": "value2"}

Example:

student = { "name": "Ankit", "age": 20, "course": "B.Tech"}

print(student)

Output:

{'name': 'Ankit', 'age': 20, 'course': 'B.Tech'}

Basic Operations on Dictionary

1. Accessing Values

Values can be accessed using their keys.

Example:

student = { "name": "Ankit", "age": 20, "course": "B.Tech"}

print(student["name"])

print(student["age"])

Output:

Ankit

20

The get() method can also be used to access values using keys.

Example:

print(student.get("course"))

Output:

B.Tech

2. Adding Elements

A new key-value pair can be added by assigning a value to a new key.

Example:

student = {"name": "Ankit", "age": 20}student["course"] = "B.Tech"

print(student)

Output:

{'name': 'Ankit', 'age': 20, 'course': 'B.Tech'}

3. Updating Elements

The value of an existing key can be changed by assigning a new value.

Example:

student = {"name": "Ankit", "age": 20}

student["age"] = 21print(student)

Output:

{'name': 'Ankit', 'age': 21}

The update() method can also update multiple key-value pairs.

Example:

student.update({"age": 22, "course": "B.Tech"})

print(student)

4. Removing Elements

Method	**Description**
pop()	Removes the specified key and returns its value.
popitem()	Removes and returns the last inserted key-value pair.
clear()	Removes all elements from the dictionary.

Example:

student = { "name": "Ankit", "age": 20, "course": "B.Tech"}

student.pop("age")

print(student)

student.popitem()

print(student)

student.clear()

print(student)

Output:

{'name': 'Ankit', 'course': 'B.Tech'}

{'name': 'Ankit'}

{}

5. Dictionary Methods

Method	Description
keys()	Returns all keys.
values()	Returns all values.
items()	Returns all key-value pairs.
get()	Returns the value of a specified key.
update()	Updates the dictionary.
len()	Returns the number of key-value pairs.

Example:

```
student = {    "name": "Ankit",    "age": 20,    "course": "B.Tech"}
print(student.keys())
print(student.values())
print(student.items())
print(len(student))
```

Output:

```
dict_keys(['name', 'age', 'course'])
dict_values(['Ankit', 20, 'B.Tech'])
dict_items([('name', 'Ankit'), ('age', 20), ('course', 'B.Tech')])
3
```

6. Iterating Through a Dictionary

A for loop can be used to access dictionary keys and values.

Example:

```
student = {    "name": "Ankit",    "age": 20,    "course": "B.Tech"}
for key, value in student.items():
        print(key, ":", value)
```

Output:

```
name : Ankit
age : 20
course : B.Tech
```

4. Set in Python

A set is an unordered collection of unique elements. It does not allow duplicate values. Sets are useful for performing mathematical operations such as union, intersection and difference.

Syntax:

set_name = {element1, element2, element3}

Example:

numbers = {10, 20, 30, 40, 50}print(numbers)

Output:

{10, 20, 30, 40, 50}

Note: Sets do not maintain a fixed order, so the output order may vary.

Basic Operations on Set

1. Adding Elements

The add() method adds a single element, whereas update() adds multiple elements.

Example:

numbers = {10, 20, 30}numbers.add(40)print(numbers)numbers.update([50, 60])print(numbers)

Output:

{10, 20, 30, 40}

{10, 20, 30, 40, 50, 60}

2. Removing Elements

Method	Description
remove()	Removes the specified element. Gives an error if the element is absent.
discard()	Removes the specified element without an error if absent.
pop()	Removes and returns an arbitrary element.
clear()	Removes all elements.

Example:

numbers = {10, 20, 30, 40}numbers.remove(20)print(numbers)numbers.discard(50)print(numbers)numbers.pop()print(numbers)numbers.clear()print(numbers)

Note: The element removed by pop() is not fixed because sets are unordered.

3. Union Operation

Union combines all unique elements of two sets.

Operator: |

Method: union()

Example:

A = {10, 20, 30}B = {30, 40, 50}print(A | B)

Output:

{10, 20, 30, 40, 50}

4. Intersection Operation

Intersection returns only the common elements present in both sets.

Operator: &

Method: intersection()

Example:

A = {10, 20, 30}B = {30, 40, 50}print(A & B)

Output:

{30}

5. Difference Operation

Difference returns elements that are present in the first set but not in the second set.

Operator: -

Method: difference()

Example:

A = {10, 20, 30}B = {30, 40, 50}print(A - B)

Output:

{10, 20}

6. Symmetric Difference

Symmetric difference returns elements that are present in either set but not in both sets.

Operator: ^

Method: symmetric_difference()

Example:

A = {10, 20, 30}B = {30, 40, 50}print(A ^ B)

Output:

{10, 20, 40, 50}

7. Other Set Methods

Method	Description
len()	Returns the number of elements.
copy()	Creates a copy of the set.
issubset()	Checks whether all elements of one set are present in another set.
issuperset()	Checks whether a set contains all elements of another set.
isdisjoint()	Checks whether two sets have no common elements.

Example:

A = {10, 20}B = {10, 20, 30, 40}print(A.issubset(B))print(B.issuperset(A))print(A.isdisjoint(B))

Output:

True

True

False

5. Difference Between List, Tuple, Dictionary and Set

Feature	List	Tuple	Dictionary	Set
Symbol	[ ]	( )	{key: value}	{ }
Order	Ordered	Ordered	Insertion ordered	Unordered
Duplicates	Allowed	Allowed	Keys are unique	Not allowed
Modification	Allowed	Not allowed	Allowed	Allowed
Access	Index	Index	Key	No indexing
Example	[1, 2, 3]	(1, 2, 3)	{"a": 1}	{1, 2, 3}

Important Note:

- An empty list is created using [].
- An empty tuple is created using ().

- An empty dictionary is created using {}.
- An empty set is created using set().

File Handling in Python

File handling is used to create, read, write and modify files using Python. It allows us to store data permanently in files.

Python provides built-in functions such as open(), read(), write() and close() for file handling.

1. Opening a File

The open() function is used to open a file.

Syntax:

file = open("filename.txt", "mode")

Common File Modes:

Mode	Description
r	Opens a file for reading.
w	Opens a file for writing. Creates a new file or overwrites existing content.
a	Opens a file for appending data at the end.
r+	Opens a file for both reading and writing.

2. Writing to a Text File

The write() function is used to write data into a text file.

Program:

file = open("student.txt", "w")file.write("Welcome to Python Programming")file.close()

Explanation (Line by Line):

Line 1:

file = open("student.txt", "w")

- open() opens the file named student.txt.
- "w" represents write mode.
- If the file does not exist, Python creates it.
- If the file already exists, its previous content is overwritten.
- The file object is stored in the variable file.

Line 2:

file.write("Welcome to Python Programming")

- write() writes the given text into the file.
- The text is stored in student.txt.
- It does not automatically add a new line.

Line 3:

file.close()

- close() closes the opened file.
- It ensures that the file is properly closed and pending changes are saved.

Content of student.txt:

Welcome to Python Programming

3. Reading from a Text File

The read() function is used to read the contents of a text file.

Program:

file = open("student.txt", "r")data = file.read()print(data)file.close()

Output:

Welcome to Python Programming

Explanation (Line by Line):

Line 1:

file = open("student.txt", "r")

- Opens the file student.txt.
- "r" represents read mode.
- The file must already exist; otherwise, Python raises a FileNotFoundError.

Line 2:

data = file.read()

- read() reads the contents of the file.
- The complete content is returned as a string.
- The content is stored in the variable data.

Line 3:

print(data)

- Displays the content stored in the variable data.

Line 4:

file.close()

Closes the opened file.

4. Reading a CSV File

CSV stands for Comma-Separated Values. It is a file format used to store tabular data in rows and columns.

Python provides the built-in csv module to read and write CSV files.

Example of student.csv:

Name,Age,Course

Ankit,20,B.Tech

Rahul,21,BCA

Program to Read a CSV File:

import csvfile = open("student.csv", "r")reader = csv.reader(file)for row in reader: print(row)file.close()

Output:

['Name', 'Age', 'Course']

['Ankit', '20', 'B.Tech']

['Rahul', '21', 'BCA']

Explanation (Line by Line):

Line 1:

import csv

- Imports the built-in csv module.
- This module provides functions to work with CSV files.

Line 2:

file = open("student.csv", "r")

- Opens the file named student.csv.
- "r" represents read mode.
- The file object is stored in the variable file.

Line 3:

reader = csv.reader(file)

- csv.reader() reads the CSV file.
- It separates each row into individual values using commas.
- The result is stored in the variable reader.

Line 4:

for row in reader:

- Uses a for loop to access each row of the CSV file one by one.
- Each row is returned as a list.

Line 5:

print(row)

- Displays each row of the CSV file.
- Each row is printed as a list of strings.

Line 6:

file.close()

Closes the CSV file after reading.

5. Important File Handling Functions

Function	Description
open()	Opens a file.
read()	Reads the complete file content.
readline()	Reads one line from a file.
readlines()	Reads all lines and returns them as a list.
write()	Writes data into a file.
writelines()	Writes multiple strings into a file.
close()	Closes the file.

Note: File handling is important because it allows Python programs to store and retrieve data permanently.